

AI HW1

Charles Timmes, Tome Nedeski, Cindy Zhuang

February 22nd 2026

1 Part 0

1.1 Maze Generation

To generate the maze we implemented a DFS algorithm with random tie-breaking and backtracking. The process begins by initializing a 101x101 grid of all zeros, which are unblocked cells and randomly selects a starting position. we maintain three key data structures, a visited array to track cells that have been explored, a stack for backtracking, and a the grid itself where we record blocked and unblocked cells. Note that blocked cells are set equal to 1. At each iteration, we examine the four neighbors, north, south, east, and west of the current cell using a `get_neighbors()` helper function. we filter these neighbors to find unvisited cells. If unvisited neighbors exist, we randomly select one using the `next_neighbor()` function, mark it as visited and probabilistically determine if the cell is blocked. Each cell has a probability of 30% of being blocked. We only add unblocked cells to the backtracking stack, this ensures that our DFS path consists of only traversable cells.

When we come across a dead end, we enter the backtracking phase. We repeatedly pop cells from the stack until we find one with unvisited neighbors. This continues the DFS exploration from a different branch. If the stack becomes empty but unvisited cells remain, we select any unvisited cell, mark it as unblocked, add it to the stack and resume the algorithm. This ensures all cells are eventually visited. The algorithm terminates when all 10,201 cells have been visited, guaranteeing a fully connected grid with no isolated regions. The use of random neighbor selection and probabilistic blocking creates a maze like structure with corridors and chambers rather than randomly scattered obstacles.

1.2 Maze Statistics

Mean	Std Dev	Max	Min
29.86%	0.42%	30.7%	28.93%

1.3 Sample Grid Visualizations

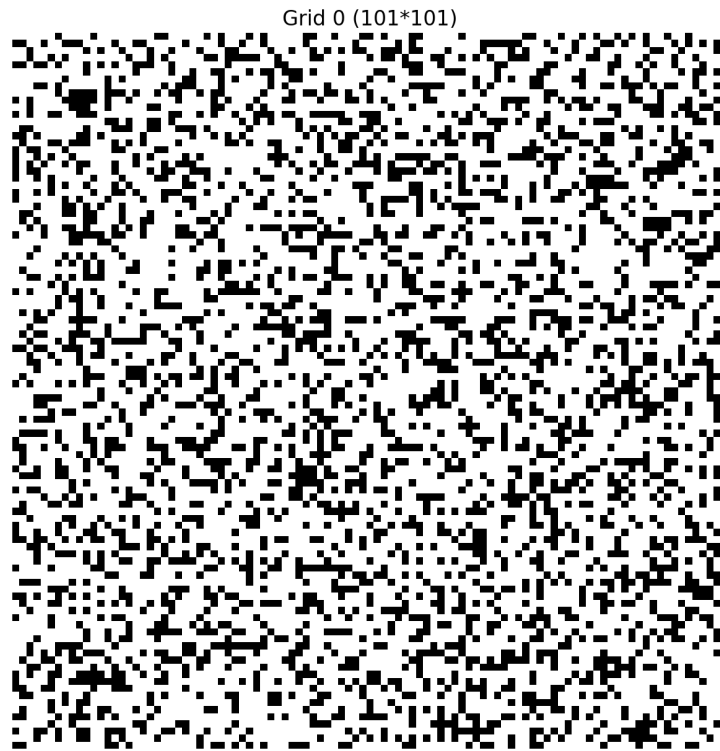


Figure 1: Grid 0

Grid 1 (101*101)

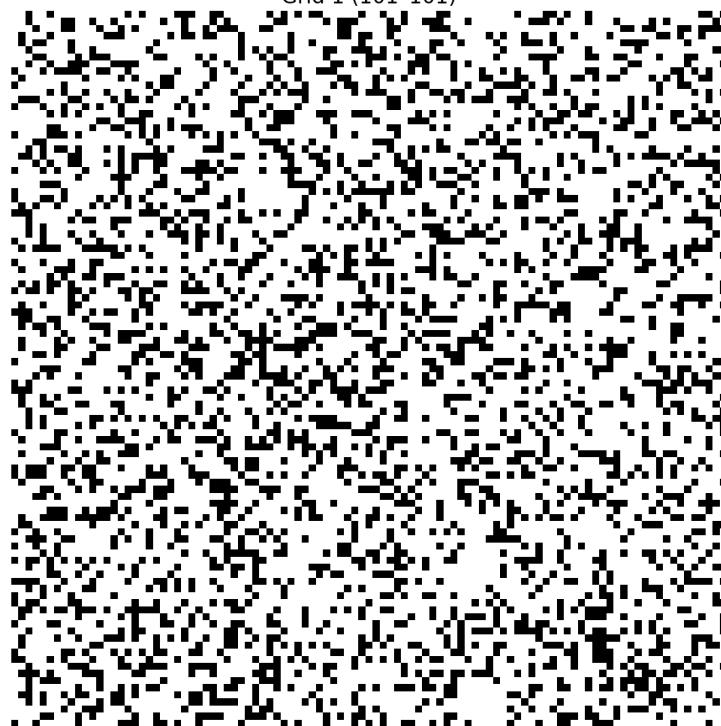


Figure 2: Grid 1

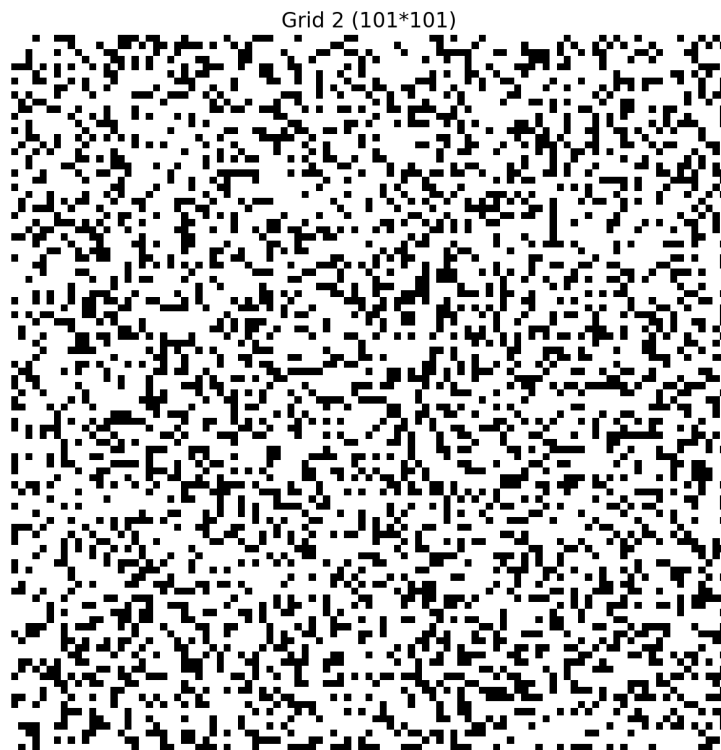


Figure 3: Grid 2

2 Part 1

2.1 a)

To first understand why the agent will move to the east let us define our current cell and the neighboring cells. Our current cell is at coordinate $(2, 5)$. The neighboring cells are the northern at coordinate $(2, 4)$, eastern at coordinate $(3, 5)$, and western at coordinate $(1, 5)$. Note that the southern cell is out of bounds. Now that we have defined our neighbors, we will let $g(n)$ for each movement be 1. As defined in the assignment the $h(n)$ will be the manhattan distance. We will calculate the manhattan distances below.

1. The Eastern $h(n) = |3 - 5| + |5 - 5| = 2$, $h(n) + g(n) = 3$
2. The Northern $h(n) = |2 - 5| + |4 - 5| = 4$, $h(n) + g(n) = 5$
3. The Western $h(n) = |1 - 5| + |5 - 5| = 4$, $h(n) + g(n) = 5$

Looking at the calculation of $f(n)$ for our current cell and its neighbors we can see that based off of the A^* algorithm the next movement for the shortest

path will be selected as the cell to the east, since we have a value of 3 for this path. In addition it is good to remember that the agent does not know what cells are blocked and assumes each cell is unblocked. Thus providing further reasoning as to why the eastern cell will be selected.

2.2 b)

There are two possible outcomes in this search problem, the agent reaches the target, or the agent determines it is impossible. In the first case, the agent successfully follows shortest presumed-unblocked paths found by A^* until it arrives at the target. In the second case, the agent knows it is impossible when the OPEN list becomes empty during a search. This occurs when all reachable unblocked cells from the agent's current position have been explored, there may still be unblocked cells on the other side of blocked cells, but A^* has exhausted all reachable possibilities. The algorithm must converge to one of these outcomes in finite time because the grid contains a finite number of cells and A^* never expands the same cell twice within a single search. Thus each search terminates. Furthermore, the agent only replans when it discovers that its current path is blocked and it can only discover blocked cells by physically moving to an adjacent cell and observing its neighbors. Since there are finitely many blocked cells to discover, the number of replans is finite and the overall process must terminate.

For the n^2 bound let n be the number of unblocked cells. We are counting the agent's physical moves through the gridworld. The agent only replans when it discovers a blocked cell on its current path, which requires the agent to have moved at least one cell since the last replan. Therefore the maximum number of replans is n , since the agent can move to at most n unblocked cells total. Additionally, the maximum path length after any replan is n , since a path cannot visit more cells than the total number of unblocked cells. Thus the total number of moves $\leq n$ replans $\times n$ moves per path $= n^2$

3 Part 2

We implemented Repeated Forward A^* with two different tie-breaking strategies. The first strategy breaks ties in favor of cells with larger g -values, while the second breaks ties in favor of cells with smaller g -values. We ran both versions on all 50 gridworlds using the same fixed start and goal positions for each grid to ensure a fair comparison. We measured the total number of expanded cells across all A^* searches for each run.

Our results showed that breaking ties in favor of larger g -values consistently expanded fewer cells. Out of 21 solvable grids, the larger g strategy expanded fewer cells in every single case. On average, the smaller g strategy expanded about 28.5 times more cells than the larger g strategy (277,996 vs 9,761 average expanded cells). The remaining 29 grids were unsolvable for both strategies, meaning the start was separated from the goal by blocked cells.

The reason for this comes down to how each strategy explores the grid. When we break ties in favor of larger g -values, the search behaves more like a depth-first search. Among cells with the same f -value, it prefers the one that is farther from the start and therefore closer to the goal. This causes the search to push deep along a narrow path toward the goal, expanding fewer cells overall. On the other hand, breaking ties in favor of smaller g -values causes the search to behave more like a breadth-first search. It prefers cells closer to the start, which means it spreads out in a wide frontier and explores many cells that are not on the optimal path. This leads to significantly more expansions before reaching the goal.

4 Part 3

In this part, we compared **Repeated Forward A*** and **Repeated Backward A*** on the same set of 50 generated 101×101 gridworlds. Repeated Forward A* replans by running A* from the agent’s *current position* to the *fixed target*. Repeated Backward A* replans by running A* from the *target* to the agent’s *current position*, and then the agent follows the reversed path toward the target.

To ensure a fair comparison, both methods used the same tie-breaking strategy: among nodes with the same

$$f(s) = g(s) + h(s),$$

we expanded the node with **larger** $g(s)$ first, and any remaining ties were resolved identically.

We measured:

- **Total expanded cells:** the sum of expanded nodes across all A* calls during repeated replanning.

Among the 50 gridworlds, both algorithms reached the goal in 21/50 cases. On these comparable instances, the average performance is shown in Table 1.

Metric (both reached)	Forward	Backward
Number of comparable grids	21/50	
Avg. expanded cells	9761.5	121947.4

Table 1: Part 3

Overall, Forward expanded far fewer cells (about $\approx 12.5\times$ fewer) than Backward on average.

Even with identical tie-breaking, the **direction of search** can significantly change the explored region in repeated replanning. In Forward A*, the heuristic $h(s)$ is always computed relative to the *fixed target*, so the heuristic landscape remains stable across replans and the search tends to stay goal-directed as the agent approaches the target. In Backward A*, the A* goal becomes the *moving agent position* each time replanning occurs; this shifting reference point can

lead to less consistent focusing behavior and substantially more re-expansion of states, especially in corridor-like maps with many branches. This explains the large observed gap in expanded cells.

5 Part 4

Let us begin by first defining a few variables, let n be a cell, let $h(n)$ be the manhattan distance to our target cell, let n' be the neighbor of cell n . To prove consistency we must satisfy the following condition: $h(n) \leq c(n, a, n') + h(n')$. We can simplify this further to be $Manhattan(n, goal) \leq 1 + Manhattan(n', goal)$, note that $c(n, a, n') = 1$. We will show this condition to be true by first recalling that the agent can only move in 4 directions, north, east, south, west. Given this we can say that in each case there will be a change in $h(n)$ to $h(n')$ by at most 1. Since each cardinal move changes exactly one coordinate by exactly 1 and Manhattan distance is the sum of absolute coordinate differences, a single step can change the manhattan distance by at most 1. Formally we can define $h(n') \in \{h(n) - 1, h(n), h(n) + 1\}$. Let us now examine each of the these cases in which $h(n')$ can take by using the condition above.

1. Case 1: $h(n') = h(n) - 1 \rightarrow h(n) \leq h(n)$
2. Case 2: $h(n') = h(n) \rightarrow h(n) \leq h(n) + 1$
3. Case 3: $h(n') = h(n) + 1 \rightarrow h(n) \leq h(n) + 2$

All three cases satisfy the $h(n) \leq 1 + h(n')$ condition, therefore the consistency condition holds true. Since any value is less than or equal to itself, and strictly less than itself plus a positive constant.

Let us represent our new heuristic value using the consistency condition $h_{new}(s) \leq c(s, a, s') + h_{new}(s')$. If we recall from the instructions we can see that $h_{new} = g(s_{goal}) - g(s)$. If we plug this into our conditions above we get the following: $g(s_{goal}) - g(s) \leq c(s, a, s') + g(s_{goal}) - g(s')$. We can cancel out the $g(s_{goal})$ and we will be left with the following: $g(s') \leq c(s, a, s') + g(s)$ to be our reduced condition of consistency. In this case $g(s)$ is the actual cost to reach s from the start along the path A^* found. If we have found an optimal path to s with cost $g(s)$ then one way to reach s' is to go to s first and then take action a . That path would cost $g(s) + c(s, a, s')$. Thus the optimal path to s' must be less than or equal to any valid path to s' . Note that this inequality holds regardless of the value of $c(s, a, s')$, so consistency is maintained even when action costs increase. This proof addresses the case where both s and s' were expanded. For states not expanded, they retain their original consistent h -values and since $h_{new}(s) \geq h(s)$ for expanded states, consistency is preserved across the boundary between updated and non-updated states. When action costs increase between searches, the true goal distances can only increase or stay the same. Since $h_{new}(s)$ was computed from a previous search where costs were lower or equal, we have $h_{new}(s) \leq$ the true goal distance under the old costs $\leq$ the true goal distance under the new costs. Therefore h_{new} remains

admissible, and since the triangle inequality $g(s') \leq c(s, a, s') + g(s)$ holds for any non-negative cost values, the consistency argument above still applies even after costs increase. Therefore the consistency condition holds.

6 Part 5

In this part, we compared **Repeated Forward A*** and **Adaptive A*** on the same set of 50 generated 101×101 gridworlds. Both algorithms use repeated replanning: at each step the agent plans under its current knowledge and replans whenever it discovers blocked cells along the intended path. Adaptive A* differs from Repeated Forward A* in that it updates heuristic values after each A* search to reuse information in later searches.

To ensure a fair comparison, both methods used the same tie-breaking strategy: among nodes with the same

$$f(s) = g(s) + h(s),$$

we expanded the node with **larger** $g(s)$ first, and any remaining ties were resolved identically.

We measured:

- **Total expanded cells:** the sum of expanded nodes across all A* calls during repeated replanning.

Across the 50 gridworlds, both algorithms reached the goal in 21/50 cases (the remaining 29 cases were blocked for both methods). On the solvable instances, Adaptive A* expanded fewer cells than Repeated Forward A* on 16 out of the 21 grids. Table 2 summarizes average performance over the 21 comparable (solvable) runs.

Metric (both reached)	Repeated Forward	Adaptive A*
Comparable grids	21/50	
Avg. expanded cells	9761.5	9769.4

Table 2: Part 5: Repeated Forward A* vs Adaptive A* (averages computed over the 21 solvable grids). Adaptive A* expanded fewer cells on 16 of these 21 grids.

6.1 Discussion

Overall, Adaptive A* achieved a modest reduction in search effort on this dataset. While it expanded fewer cells on most solvable grids, the average improvement in expanded cells is relatively small. This is expected because both approaches perform repeated A* searches on the same grid sizes, and the benefit of Adaptive A* depends on how much heuristic information from earlier searches transfers to later replans. In some maps, Repeated Forward A* is already highly goal-directed, leaving limited room for additional savings.

7 Part 6

To determine whether the performance difference between Repeated Forward A^* with larger g tie-breaking and smaller g tie-breaking is statistically significant, we can use a paired t-test. We choose a paired test because both algorithms are run on the same 50 gridworlds with the same start and goal positions, so the results are naturally paired.

For each of the 50 solvable gridworlds, we record the total number of expanded cells for both tie-breaking strategies. This gives us 50 pairs of observations. We then compute the difference in expanded cells for each pair. The null hypothesis is that the mean difference is zero, meaning any observed performance gap is due to random variation in the test cases. The alternative hypothesis is that the mean difference is not zero, meaning the performance gap is systematic.

We would compute the mean and standard deviation of the 50 differences, then calculate the t-statistic as the mean difference divided by the standard error (standard deviation divided by $\sqrt{50}$). We compare this t-statistic to a t-distribution with 49 degrees of freedom. If we set our significance level at $\alpha = 0.05$ and the resulting p-value is less than 0.05, we reject the null hypothesis and conclude that the performance difference is statistically significant and not just due to sampling noise.